COMPILER CONSTRUCTION LAB · WS 2026/27 · PA1

A ChocoPy front-end: lexing and parsing

JFlex scanner · CUP LALR(1) parser · JSON AST with error recovery

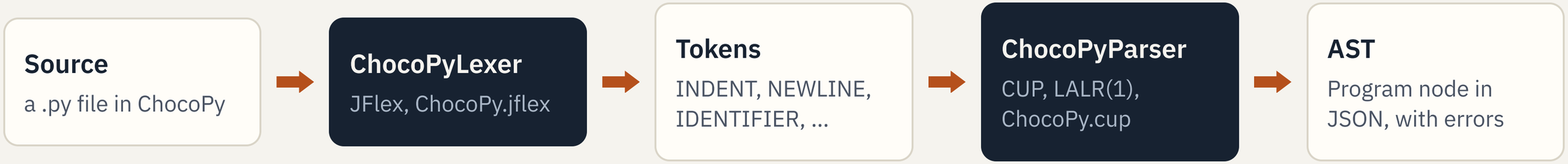
Group 4

Omar Kamel · Malak Chaouch

University of Passau
Chair of Software Engineering I

OVERVIEW

From ChocoPy source to a JSON syntax tree



33 / 33

sample tests pass, plus our own good.py and bad.py

11,329

programs parsed exactly like the reference parser

0

conflicts left in the grammar after precedences

The lexer works in two states

YYINITIAL

Start of a physical line: skip blank and comment-only lines, measure the indentation, emit INDENT or DEDENT.

first real character of the line

line break, emits NEWLINE

LINE_BODY

Inside a logical line: keywords, identifiers, literals and operators; spaces and comments are skipped.

Maximal munch, then rule order

classic is one IDENTIFIER, **class** is a keyword: keyword rules come before the identifier rule.

Reserved Python keywords

break, **import**, **lambda** and 13 more become RESERVED: never an identifier, never in a grammar rule.

The lexer never fails

Unknown input such as **$** or **&** becomes UNRECOGNIZED, so the parser reports it with its position.

Indentation: one rule, a stack and yypushback

```
{WhiteSpace}* [^ \t\r\n#] {
  yypushback(1);          // keep the first char
  int indentation = indentationOf(yytext());
  if (indentation > indentStack.peek()) {
    indentStack.push(indentation);
    yybegin(LINE_BODY);
    return INDENT;
  }
  if (indentation < indentStack.peek()) {
    indentStack.pop();
    yypushback(width);    // fire again
    return DEDENT;
  }
  yybegin(LINE_BODY);     // same level
}
```

Deeper

push the level, one INDENT

Same level

no token, continue the line

Shallower

pop, one DEDENT, push the whitespace back: the rule fires again until the level matches

Tabs advance to the next multiple of 8. At the end of input: NEWLINE if missing, one DEDENT per open level, EOF. **ChocoPy.jflex**, lines 162–192 and 283–298

Example: the tokens of two nested blocks

Line	Source	Stack	Tokens
1	if x:	[0]	IF IDENTIFIER COLON NEWLINE
2	····if y:	[0, 4]	**INDENT** IF IDENTIFIER COLON NEWLINE
3	········a = 1	[0, 4, 8]	**INDENT** IDENTIFIER EQ INTEGER NEWLINE
4	(blank line)	[0, 4, 8]	none
5	····# done	[0, 4, 8]	none: comment-only lines are skipped
6	b = 2	[0]	**DEDENT DEDENT** IDENTIFIER EQ INTEGER NEWLINE
end		[0]	EOF

Line 6 runs the indentation rule three times: two DEDENTs, then the same level. The comment on line 5 is indented by 4, but it neither closes a block nor breaks it.

LEXER · OMAR

Literals and the edge cases we checked

Input	Token	Why
"abc"	IDSTRING	looks like an identifier, so it may name a type
"a b"	STRING	any other printable ASCII text
"He\"llo"	STRING He"llo	escapes \" \n \t \\ are applied to the value
"Hell\o"	UNRECOGNIZED	illegal escape: the whole literal is one bad token
2147483648	UNRECOGNIZED	larger than 2^31 – 1
007	INTEGER ×3	0, 0, 7: no leading zeros, maximal munch
\r, \n, \r\n	NEWLINE	all three line ends, on every platform

Every row was compared with the reference lexer. Fuzzing found one crash here: a 20-digit literal overflowed **long**; now any number that does not fit an **int** is UNRECOGNIZED.

Expressions: ambiguous rules, precedences decide

```
/* lowest to highest */
precedence right IF, ELSE;
precedence left  OR;
precedence left  AND;
precedence right NOT;
precedence nonassoc EQEQ, NEQ, LT,
           GT, LEQ, GEQ, IS;
precedence left  PLUS, MINUS;
precedence left  TIMES, IDIV, MOD;
precedence right UMINUS;
precedence left  DOT, LINDEX;

cexpr ::= cexpr PLUS cexpr | …
      | MINUS cexpr %prec UMINUS
```

As in the LR lecture: an ambiguous grammar plus precedence and associativity resolves every shift/reduce conflict.

> **`a < b < c`**
>
> error at the second <: comparisons are nonassoc

> **`True == not False`**
>
> error at not: only expr, not cexpr, may use not

> **`-a.b[0]`**
>
> is −(a.b[0]): DOT and LINDEX bind tightest

PARSER · MALAK

Building the AST: elif and exact locations

```
if a:
    x = 1
elif b:
    x = 2
else:
    x = 3
```

IfStmt condition a, thenBody [x = 1], elseBody:

> **IfStmt** condition b, thenBody [x = 2], elseBody [x = 3]
>
> starts at the elif keyword, ends at the DEDENT of the else block

Leftmost to rightmost

A node spans its CUP symbols: (1 + 2) * 3 covers columns 1–11, parentheses included.

Statements stop early

A statement node ends before its NEWLINE, but a function ends where that NEWLINE ends.

Blocks end at DEDENT

if, while and for end at the DEDENT, which spans the indentation of the next line.

ERROR RECOVERY · MALAK

Three problems with the error symbol

1 · CONFLICTS

Declaration or statement?

Both can start at the same place, so error rules on both lists clashed on error.

Fix: right-recursive bodies; exactly one rule shifts error at every position.

```
program_body ::=
  decl program_body
| recovery program_body
| stmt stmts | ε
```

2 · CASCADES

Broken headers

After an error in def f(a, b) -> 1: the indented body is left behind.

Fix: parse the body, report its errors, drop it.

```
recovery ::=
  error NEWLINE
| error NEWLINE INDENT
  skipped_body dedent
```

3 · LOST DEDENT

A bad token after a block

LR reduces a block only on a valid lookahead; recovery popped the DEDENT.

Fix: recover right after the DEDENT; precedence makes shifting error win.

```
dedent ::= DEDENT
  | DEDENT recovery
precedence left DEDENT;
precedence left error;
```

ERROR RECOVERY · MALAK

Result on our bad.py

12 / 12

planted errors reported by us, each once, on its own line

19

errors from the reference parser, with follow-ups at DEDENTs

```
x: int = 1 + 2              ← 6:12 PLUS
y: int = 3                  kept

def add(a: int, b) -> int:  ← 11:18 RPAREN
    z: int = a              ← 12:14 IDENTIFIER
    return a + b

class Point:                ← 16:12 COLON
    x: int = 0

origin: int = 0             kept
…
while x > 0                 ← 47:12 NEWLINE
lambda = 3                  ← 51:1 RESERVED
```

TESTING · OMAR

The reference parser as a test oracle

35 sample and student tests

99 hand-written edge cases

11,195 random mutants

Both parsers in one JVM

valid program: the whole JSON must be equal, locations included

invalid program: the first error at the same line and column

11,329

identical, 0 failures

Mutants

delete, insert, duplicate or swap tokens, add or remove indentation in every test file

Bugs it found in our code

a crash on huge integers, DEDENT columns at EOF, the start of a program that begins with pass

TESTING · OMAR

What we found in the reference and the material

Where	Finding
Reference lexer	No NEWLINE at end of input without a final line break; hidden because the driver appends "\n"
Reference locations	The k-th DEDENT at EOF sits at column k of a line after the file, e.g. [1, 1, 5, 3] for a 4-line file
Reference messages	"Parse error near token DEDENT: x" shows the text of the previous token
Local --test mode	Does not compare locations, so wrong locations still pass
Specification	Kind NonlocalDecl is NonLocalDecl in the AST and outputs; typo "stater code"
Manual 2.8.6	"outputs outputs its printed form"

Each item has a small input and the expected output in our README.

SUMMARY

Who did what

Omar Kamel

LEXER AND TESTING

ChocoPy.jflex: line structure, INDENT and DEDENT, tabs, string and integer literals, reserved keywords, UNRECOGNIZED; good.py; differential testing against the reference

Malak Chaouch

GRAMMAR, AST AND RECOVERY

ChocoPy.cup: declarations, statements, expressions and precedences, elif desugaring, exact node locations, error rules; bad.py

We wrote the README together and reviewed every part of each other's code, so both of us can explain the whole project. About 30 hours in total.

FOR THE Q&A

Where each part lives in the code

Part	File	Lines
Indentation stack and the tab rule	ChocoPy.jflex	83-103
Start of a line: INDENT and DEDENT	ChocoPy.jflex	162-192
Keywords, literals, operators	ChocoPy.jflex	198-279
End of input	ChocoPy.jflex	283-298
Operator precedence	ChocoPy.cup	211-224
Program body and error recovery	ChocoPy.cup	232-296
Statements, blocks and elif	ChocoPy.cup	403-462
Expressions and the AST nodes	ChocoPy.cup	464-554

PA1 · GROUP 4

Thank you. Questions?

Lexer, grammar, error recovery, testing: ask any of us.

```
mvn clean package
java -cp "chocopy-ref.jar;target/assignment.jar" chocopy.ChocoPy --pass=s --dir src/test/data/pa1 --test
```